Professional portfolio

Kerk Gerald Quinamot

Senior QA Engineer

Test design · Hands-on testing · Web & mobile automation

I lead QA work from requirements review to hands-on validation, connecting reviewed test cases to web and mobile automation. I check what the tests actually prove and make the remaining risks clear.

Core QA practice

Requirements and acceptance-criteria review · Test design and risk-based coverage · Exploratory and regression testing · Web, mobile, and release validation · API checks and defect verification · Playwright / TypeScript and Maestro · AI-assisted QA review

Professional foundation

Since 2014, my QA work has progressed through software testing roles alongside a Certified Tester Foundation Level (CTFL) credential and a degree in Information Technology. Earlier teaching experience also informs how I explain testing concepts.

My recent practice combines QA leadership, requirements review, hands-on web and mobile testing, and automation review. I work through expected behavior, edge cases, defects, and release risk, while keeping the test case connected to its implementation and result.

Certification	Certified Tester Foundation Level (CTFL), 2020
Education	Bachelor of Science, Information Technology, 2013
QA experience	QA professional since 2014
Earlier experience	Guest Lecturer, 2013-2014
Recent practice	QA leadership, hands-on testing, and automation review

Where I contribute

Test design and risk

Challenge unclear acceptance criteria, work through edge cases, and define traceable cases with meaningful expected results.

Web and mobile quality

Test behavior hands-on, identify automation candidates, and review whether scripts and results match the test intent.

QA ownership and risk

Make testing decisions and release risks clear through practical review workflows and traceable evidence.

In this portfolio: two automation examples, supporting QA practice, tools, and a separate synthetic work sample.

WEB AUTOMATION / PLAYWRIGHT + TYPESCRIPT / WEB-01

Keeping a web test traceable, from case to result.

A web-admin workflow connects a reviewed sign-in-form case to a readable BDD scenario, TypeScript assertions, and a matching saved result.

My role: Led QA direction and case review; reviewed AI-assisted scripts and performed hands-on testing.

I lead the QA work, review AI output, and perform hands-on testing. AI assists with preparation and script implementation. My contribution is QA direction, test review, and hands-on validation - not sole authorship of the scripts or frameworks.

PROBLEM

When cases, scripts, and reports drift apart, a passing result can be hard to interpret. The selected example keeps the expected behavior identifiable through each stage.

SOURCE TEST CASE

The selected case appears in the full case CSV and a lean flow CSV. Its steps and expected result describe the required sign-in controls being visible.

AUTOMATION CANDIDATE

Generated cases are reviewed and selected for automation. This case has a corresponding automated scenario; its individual selection rationale was not recorded in the inspected files.

IMPLEMENTATION AND ASSERTIONS

A Gherkin scenario invokes TypeScript step definitions. Page objects provide role- and label-based locators; the step definitions assert that the required form controls are visible.

Evidence reviewed

- Source test-case row and matching BDD scenario.
- Corresponding step definitions, assertions, and page-object locators.
- Saved workbook row matching the case title, steps, and expected result.

SAVED RESULT

The matching saved result records Pass for this case.

WHAT THIS DOES NOT ESTABLISH

This is a historical result, not verification of the current build or the whole suite. A separate archived HTML report belongs to a different, mixed-outcome run and is not combined with this result.

The useful result is not just Pass: it is a result tied to a known case and an assertion that matches its intent.

Evidence below comes from inspected test cases, implementations, and saved reports. Results are historical, not fresh runs or suite-wide pass claims. Source code and raw reports remain private.

MOBILE AUTOMATION / MAESTRO / MOBILE-01

Checking what a passing mobile flow actually proves.

A candidate register maps generated cases to platform-specific flows. A required-field example shows why the assertion matters as much as the execution status.

My role: Led QA work, reviewed automation candidates and agent output, and performed hands-on validation.

PROBLEM

The same expected behavior can be implemented differently on iOS and Android. Those checks need to be reviewed before treating the platform results as equivalent.

SOURCE TEST CASE

The flow header and candidate register identify a required-field case: the action button should remain disabled while the field is empty. The original master test-case export was not part of the inspected evidence chain.

AUTOMATION CANDIDATE

The register explicitly separates candidates ready now, candidates needing setup, and manual-only work. The selected case is marked ready for automation.

IMPLEMENTATION AND ASSERTIONS

The iOS flow directly checks that the button is disabled. The Android flow taps the button and checks that specified next screens do not appear. These are different assertions and are not presented as equivalent proof.

Evidence reviewed

- Candidate classification and identifiable case intent in the flow header.
- Separate iOS and Android Maestro flows with inspected assertions.
- Saved Android failure, retry report, and matching completed assertion records.

SAVED RESULT

The saved Android retry passes its implemented navigation checks.

WHAT THIS DOES NOT ESTABLISH

The Android checks do not directly prove the original disabled-button property, so the full expected result remains unverified there. A matching iOS execution was not reviewed. No current-build or cross-platform pass is claimed.

A green flow is not enough if the assertions prove a different behavior from the original case.

Supporting QA practice

Beyond automation.

DOCUMENTED PRACTICE / REQUIREMENTS

Acceptance-criteria review and change verification

A review process for identifying ambiguity, assigning clarification owners, and rechecking findings when the acceptance criteria change. I connect the review decisions to the cases that depend on them.

My role: Test design, requirements review, and verification

PROBLEM

A requirement can look complete while leaving critical behavior undecided. Later edits can also invalidate a finding or break the link between a scenario and its tests.

- Review criteria and design material before generating test cases.
- Assign clarification owners and distinguish blocking findings from advisory notes.
- Revisit earlier findings when criteria change, preserving their identifiers and recording the decision.
- Write BDD-style cases with traceability and visible execution dependencies.

Deliverables: AC review findings, clarification records, BDD test cases, change-verification notes.

When behavior is unclear, I ask for a decision before treating it as an expected result.

Documented practice. A specific redacted finding-to-change-to-result case is not included; no defect-prevention or productivity metric is claimed.

DOCUMENTED PRACTICE / MOBILE & RELEASE

iOS and Android design-to-build reviews

Platform comparisons that check the design against each build and distinguish defects from native behavior or an outdated reference. I combine the comparisons with release checks and fix-verification evidence.

My role: Hands-on testing, review, and defect verification

PROBLEM

A matching screen is only one part of quality. Platform conventions, existing user state, stale content, and failed connections can change the experience.

- Compare design to iOS, design to Android, and behavior across platforms.
- Distinguish expected native behavior and stale design references from defects.
- Plan checks that require an upgrade from the previous build rather than a fresh install.
- Use network-failure scenarios and confirm expected behavior when re-verifying a fix.

Deliverables: Platform comparison notes, risk-based release checklists, reproducibility records, fix-verification evidence.

I record what was checked on each platform and verify a fix against the expected behavior.

Documented practice. A result-verified release or comparison case is not included; results are not inferred from one platform to another.

Tools in context

Tools and current practice.

How I use AI in QA

Requirements and platform scope -> Reviewed test cases -> Automation candidates -> AI-assisted script draft -> QA assertion review -> Hands-on validation

A review-led workflow: AI helps prepare work; QA judgment remains with me.

AI-assisted QA practice: Claude, ChatGPT/Codex, Gemini

AI supports test-case and report drafting, report audits, automation-candidate review, and script preparation. I anchor requests in requirements, platform scope, expected assertions, and a defined output. I personally review coverage and assertions, run hands-on checks, retain traceability, and communicate evidence, limits, and release risk.

Testing and collaboration: Bruno, Postman, Figma, ClickUp, Qase, Git

For test design, review, API checks, and keeping track of testing work.

Inspected automation examples: Playwright, TypeScript, Gherkin / BDD, Maestro

The selected cases connect inspected implementations to historical saved results, with scope and limitations stated alongside each example.

Supporting mobile practice: Appium, Python, real-device testing

Supporting practice documented in my work records. The selected mobile case uses Maestro; it is not evidence of an Appium implementation or result.

Further development: CI fundamentals and performance concepts

Continuing development, not claimed specialization. Existing static CI configuration does not establish browser-test execution in CI or independent pipeline ownership.

Synthetic work sample / no production results

A small example of my test design.

This display-name feature and defect report are fictional. They illustrate test design; they are not client work or execution evidence.

EXAMPLE REQUIREMENT

A signed-in user can update their display name. Leading and trailing spaces are trimmed before validation. The saved name must contain 1 to 50 characters and remain visible after a page reload. An invalid name must not replace the saved value.

Questions before assumptions

Character counting: How are emoji and combined Unicode characters counted?

The boundary can change across browser and server implementations.

Failed save: What message and retry behavior are expected when the save request fails?

The requirement defines validation failure but leaves network failure undecided.

Concurrent edits: What should happen when the same profile is edited in two sessions?

A later save may overwrite a value without the user realizing it.

Selected cases

CASE	FOCUS	INPUT	EXPECTED RESULT
TC-01	Positive	A valid new name	Saved name remains after reload.
TC-02	Lower boundary	One character	Name is accepted and persists.
TC-03	Upper boundary	Exactly 50 ASCII characters	Name is accepted and persists.
TC-04	Negative	51 ASCII characters	New value is rejected; saved name is unchanged.
TC-05	Normalization	Spaces only	Trimmed value is empty; saved name is unchanged.

BDD example / TC-04

```
Given a saved display name exists
And I enter 51 ASCII characters as the new name
When I attempt to save the name
Then the new value is rejected
And the previously saved name remains after reload
```

Fictional defect: Saved display name reverts after page reload

Environment: Synthetic web build / test account / browser and version recorded with evidence

Steps: 1. Record the current name. 2. Save a valid different name. 3. Reload the page.

Expected: The updated name remains visible after reload.

Actual: The previously saved name is displayed after reload.

Evidence: Before-and-after capture, sanitized request/response evidence, build identifier, and reproduction frequency.

Triage: Describe the persistence impact and establish affected scope before proposing severity and priority.

Company, client, product, project, account, and original case identifiers are omitted. Portfolio case aliases are neutral. Raw source files remain private; the separate demonstration is synthetic.